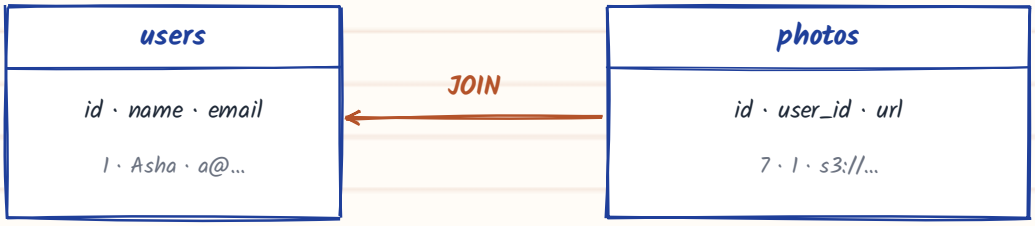
SQL vs NoSQL

Lesson 10 · choosing the right database, and saying why

In this lesson → ① The question ② SQL ③ NoSQL: 4 types ④ Trade-offs ⑤ 4 questions ⑥ Model by query ⑦ Polyglot ⑧ Myths ⑨ Interview Q&A
Running project: SnapLink, our photo-sharing app that grows from 1 server to 100M users.

1 SQL (relational)



fixed schema · joins · ACID transactions

- ✓ Tables, fixed schema, foreign keys
- ✓ Joins + ad-hoc queries (SQL)
- ✓ **ACID** multi-row transactions
- ✓ Scale: read replicas; sharding for writes

2 NoSQL: 4 families

Key-value
key → value
super fast lookups
Redis, DynamoDB

Document
JSON documents
flexible fields
MongoDB

Wide-column
rows × many columns
huge write volume
Cassandra, HBase

Graph
nodes + edges
relationships
Neo4j

Type	Model	Good for	Examples
Key-value	key → blob	cache, sessions, counters	Redis, DynamoDB
Document	JSON docs	flexible entities, catalogs	MongoDB, Couchbase
Wide-column	partition key + sorted columns	huge writes, time series, feeds	Cassandra, HBase, Bigtable
Graph	nodes + edges	social graphs, recommendations	Neo4j, Neptune

3 Trade-offs

	SQL	NoSQL
Schema	fixed, enforced	flexible (in app code)
Queries	joins, ad-hoc	by key; designed per query
Transactions	ACID multi-row	often single-item
Scaling writes	sharding (manual)	built-in horizontal
Consistency	strong	often eventual / tunable

4 4 questions to choose

1. Is the data relational? need joins? → SQL
2. Need multi-row ACID transactions? → SQL
3. Huge write volume / simple key access? → NoSQL
4. Flexible or changing schema? → document

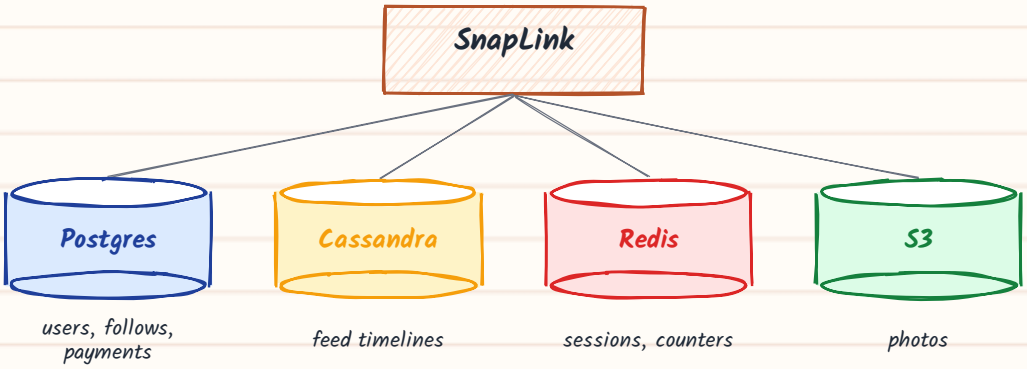
5 Model NoSQL around queries

```
CREATE TABLE feed_by_user (
  user_id  bigint,
  posted_at timestamp,
  photo_id bigint,
  author   text,
  PRIMARY KEY (user_id, posted_at)
) WITH CLUSTERING ORDER BY (posted_at DESC);
```

→ Partition key = user_id → one feed in one partition

→ Denormalise: duplicate data so reads are one lookup

6 Polyglot persistence: SnapLink



Data	Store	Why
Users, follows, payments	Postgres	relations, ACID
Feed timelines	Cassandra	huge writes, read by user + time
Sessions, counters, cache	Redis	in-memory speed
Photos	S3 (object storage)	cheap, huge blobs (SD-16)

7 Two myths

✗ "SQL can't scale": replicas, sharding, NewSQL (Spanner, CockroachDB)

✗ "NoSQL has no schema": it lives in your application

8 Interview Corner

Q. When do you choose SQL?

A. Relational data with joins and multi-row ACID transactions (payments, orders, inventory).

Q. When do you choose NoSQL?

A. Massive scale, simple key-based access, flexible schema or very high write throughput.

Q. Name the NoSQL families.

A. Key-value, document, wide-column, graph.

Q. How do you model data in Cassandra?

A. Around queries: choose partition and clustering keys per query; denormalise.

Q. What is polyglot persistence?

A. Using different databases for different parts of the system based on their needs.

⚡ Revise in 60 seconds

- → **SQL**: relations, joins, ACID
- → **KV**: fast lookups
- → **Document**: flexible JSON
- → **Wide-column**: huge writes
- → **Graph**: relationships
- → Model NoSQL by **query**
- → Polyglot persistence
- → Choose by access patterns

Next lesson → SD-11
Indexes deep dive: B-tree vs LSM-tree.
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**